# Walkthrough Demonstration Guide

Procedure, expected results, and verification for the recorded walkthrough

CS 454/554 — Introduction to Cloud Computing, Fall 2026 (graduate section)

October 2026 • Repository: https://github.com/RemyMartinRosenhoover/CS-554-MidTerm

## 1 Purpose and scope

This is the procedure I follow to record the four-minute walkthrough, and it is written so that a reader can reproduce the demonstration on an environment built from the README. Each step gives the command I enter, the result I expect, and the check I make before moving on. The expected results are the outputs I observed when I rehearsed this procedure on 19 October 2026 against the running environment; they are shown unedited apart from trimming one over-long line.

The guide assumes the environment the README describes, built and deployed as far as Part 3: the VM is running, the Compose service is up on port 8080, and the k3d cluster holds the two-replica Deployment, its Service, and the client Pod. It demonstrates what Parts 2–4 of the README establish; it does not repeat the CS 554 readiness experiment, which is documented in the README's CS 554 section and in `evidence/04-readiness-554.md`.

**Conventions.** Commands marked *VM* are entered in an SSH session inside `cs554-vm`; commands marked *host* are entered in Windows PowerShell on the machine that runs VMware. Blocks are pasted as shown, without prompt characters. Some values are live and will differ from the rehearsal; I read them from the screen rather than from this page:

| Live value | Where it appears |
|---|---|
| Guest address (`192.168.197.x`) | step 2, host request; found in precondition P1 |
| Pod names (`cs554-midterm-<hash>-<id>`) | steps 3–5 |
| Ages, request counters | every output; counters rise by one per request |
| Recovery attempt number | step 5, `recovered on attempt N` |

**Windows.** The README open in a browser (steps 1 and 6); *Terminal 1*, an SSH session at `~/CS-554-MidTerm/compose`; *Terminal 2*, an SSH session at `~/CS-554-MidTerm`; one host PowerShell window. Only P1, P6, and the host request in step 2 run on the host; every other block runs inside the VM. Each SSH session is opened from its own PowerShell window with the README's connection line:

```
$work = Join-Path $env:USERPROFILE 'cs554-work'
$keys = Join-Path $work 'keys'
$key = Join-Path $keys 'cs554_vm'
$knownHosts = Join-Path $keys 'known_hosts'
$ip = Get-Content -LiteralPath (Join-Path $work 'logs\cs554-vm-ip.txt')
ssh -i $key -o "UserKnownHostsFile=$knownHosts" "student@$ip"
```

## 2 Preconditions

I run these checks in order and do not start until all seven pass. A failure is corrected with the restoration steps in Section 4, after which the checks are run again from P1.

### P1 — The VM is running and has an address (host).

```
$vmrun = "${env:ProgramFiles(x86)}\VMware\VMware Workstation\vmrun.exe"
& $vmrun -T ws list
& $vmrun -T ws getGuestIPAddress C:\VMs\cs554-vm\cs554-vm.vmx
```

Expected: the list names `C:\VMs\cs554-vm\cs554-vm.vmx`, and the address is on `192.168.197.0/24`. I note the address; step 2 uses it.

### P2 — The checkout is at the current `main` (VM).

```
cd ~/CS-554-MidTerm
git status --porcelain
git fetch -q origin && git rev-parse --short HEAD origin/main
```

Expected: `git status` prints nothing, and the two hashes are identical. If they differ, `git pull -ff-only origin main` and repeat.

### P3 — The Compose service is healthy (VM).

```
docker inspect -f '{{.State.Health.Status}}' compose-web-1
curl -s localhost:8080/health
```

Expected: `healthy`, then `{"status":"ok"}`.

### P4 — The cluster is at its baseline (VM).

```
kubectl get deploy cs554-midterm
kubectl get pods -l app=cs554-midterm
kubectl get pod client
kubectl get svc cs554-midterm -o jsonpath='{.spec.selector.app}'; echo
kubectl get endpointslices -l kubernetes.io/service-name=cs554-midterm -o json \
  | jq '[.items[].endpoints[]? | select(.conditions.ready == true)] | length'
for p in $(kubectl get pods -l app=cs554-midterm -o name); do
    kubectl exec "$p" -- test -f /tmp/ready && echo "$p readiness file present"
done
```

Expected: Deployment `2/2`; two application Pods `1/1 Running` with `RESTARTS 0`; `client` `Running`; selector `cs554-midterm` (not `cs554-midterm-typo`); ready endpoints `2`; both Pods report the readiness file.

### P5 — No forwarder is left over (VM).

```
pgrep -af 'kubectl.*port[-]forward' || echo "no forwarder"
ss -ltn '( sport = :18080 )'
```

Expected: `no forwarder`, and the `ss` table contains only its header line.

### P6 — The host reaches the service (host, with the address from P1).

```
curl.exe -i http://<address>:8080/health
```

Expected: `HTTP/1.1 200 OK` with body `{"status":"ok"}`.

**P7 — The recording setup is ready.** The four windows are arranged inside the capture region with a readable font; nothing personal is visible (no taskbar, notifications, other tabs, or host paths); a five-second test clip has been recorded and played back with audible audio.

## 3 Procedure

Timings are the on-camera budget: 20, 50, 50, 40, 60, and 20 seconds, four minutes in total.

### Step 1 — Opening (0:00–0:20)

Screen: the README in the browser, scrolled to the repository layout and the environment summary. No command.

> **Talking points.** This is one small service packaged as one image. Compose and Kubernetes both run inside this Linux VM on Windows, so the host, the VM, the container, and the application are separate layers.

### Step 2 — Compose (0:20–1:10)

Purpose: show the service running under Compose, reached from inside the VM and from the host, with its per-process counter.

**Enter** (VM, Terminal 1):

```
cd ~/CS-554-MidTerm/compose
docker compose ps
curl -s localhost:8080/
curl -s localhost:8080/
```

**Enter** (host, with the address from P1):

```
curl.exe -s http://<address>:8080/
```

**Observed in rehearsal:**

```
NAME            IMAGE               COMMAND                  SERVICE   STATUS                  PORTS
compose-web-1   cs554-midterm:1.1   "docker-entrypoint.s..."   web       Up 20 hours (healthy)   0.0.0.0:8080->8080/tcp, ...
{"service":"cs554-compose","message":"Hello from Compose inside the VM","hostname":"9577b9f64fc0","requestCount":2}
{"service":"cs554-compose","message":"Hello from Compose inside the VM","hostname":"9577b9f64fc0","requestCount":3}
{"service":"cs554-compose","message":"Hello from Compose inside the VM","hostname":"9577b9f64fc0","requestCount":4}
```

The third response came from the host. The service's JSON has no trailing newline, so consecutive responses run together on one line in the terminal; they are separated here for readability, and the `ps` output is trimmed to the columns that matter.

> **Talking points.** The Windows host reaches the VM's NAT address directly; no port forwarding is configured. The hostname stays the same and the count rises by one with every request, because this counter belongs to that one application process.

**Check before continuing.** the host's count is exactly one more than the VM's last. If `curl.exe` cannot connect, the guest address has changed: confirm it with P1 and repeat the host line.

### Step 3 — Kubernetes (1:10–2:00)

Purpose: show the same image running as two replicas behind a Service, with requests reaching both.

**Enter** (VM, Terminal 2):

```
cd ~/CS-554-MidTerm
kubectl get deploy,pods,svc
kubectl exec client -- sh -c 'for i in 1 2 3 4; do curl -s http://cs554-midterm/; echo; done'
```

**Observed in rehearsal:**

```
NAME                              READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/cs554-midterm   2/2     2            2           22h

NAME                                  READY   STATUS    RESTARTS   AGE
pod/client                            1/1     Running   0          22h
pod/cs554-midterm-67c66d7b5d-98k98   1/1     Running   0          19h
pod/cs554-midterm-67c66d7b5d-l2ffr   1/1     Running   0          19h

NAME                    TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)   AGE
service/cs554-midterm   ClusterIP   10.43.197.200   <none>        80/TCP    22h
service/kubernetes      ClusterIP   10.43.0.1       <none>        443/TCP   23h
{"service":"cs554-k8s","message":"Hello from Kubernetes inside the VM","hostname":"cs554-midterm-67c66d7b5d-l2ffr","requestCount":45}
{"service":"cs554-k8s","message":"Hello from Kubernetes inside the VM","hostname":"cs554-midterm-67c66d7b5d-l2ffr","requestCount":46}
{"service":"cs554-k8s","message":"Hello from Kubernetes inside the VM","hostname":"cs554-midterm-67c66d7b5d-98k98","requestCount":30}
{"service":"cs554-k8s","message":"Hello from Kubernetes inside the VM","hostname":"cs554-midterm-67c66d7b5d-l2ffr","requestCount":47}
```

The four responses name two different Pods with independent counters. A short sample can land on one Pod only; that is a property of random backend selection, not a fault, and I say so if it happens.

> **Talking points.** Kubernetes uses the same image, imported with `k3d image import`, and keeps two ready replicas. These in-cluster requests go through Service DNS and ClusterIP routing; this sample reached both hostnames with independent counters, which demonstrates multiple backends, not round-robin ordering.

**Check before continuing.** all four requests returned JSON and none printed a `curl:` error.

### Step 4 — Pod replacement (2:00–2:40)

Purpose: delete one Pod and show the ReplicaSet restoring the declared count without any further apply.

**Enter** (VM, Terminal 2; the first block selects a live Pod and watches, the second runs after `Ctrl+C`):

```
pod_to_delete=$(kubectl get pods -l app=cs554-midterm -o jsonpath='{.items[0].metadata.name}')
printf 'Selected Pod: %s\n' "$pod_to_delete"
kubectl delete pod "$pod_to_delete" --wait=false
kubectl get pods -w


kubectl rollout status deploy/cs554-midterm --timeout=120s
kubectl get pods
```

**Observed in rehearsal:**

```
Selected Pod: cs554-midterm-67c66d7b5d-98k98
pod "cs554-midterm-67c66d7b5d-98k98" deleted from default namespace
NAME                              READY   STATUS        RESTARTS   AGE
client                            1/1     Running       0          22h
cs554-midterm-67c66d7b5d-98k98   1/1     Terminating   0          19h
cs554-midterm-67c66d7b5d-l2ffr   1/1     Running       0          19h
cs554-midterm-67c66d7b5d-2v5mx   0/1     Pending       0          0s
```

```
cs554-midterm-67c66d7b5d-2v5mx   0/1     ContainerCreating   0          0s
cs554-midterm-67c66d7b5d-2v5mx   0/1     Running             0          1s
cs554-midterm-67c66d7b5d-2v5mx   1/1     Running             0          7s
deployment "cs554-midterm" successfully rolled out
NAME                             READY   STATUS    RESTARTS   AGE
client                           1/1     Running   0          22h
cs554-midterm-67c66d7b5d-2v5mx   1/1     Running   0          10s
cs554-midterm-67c66d7b5d-l2ffr   1/1     Running   0          19h
```

The replacement carries a new generated name and reached `1/1` seven seconds after the deletion; repeated `Completed` and `Pending` rows from the watch are omitted above.

**Talking points.** The ReplicaSet controller sees fewer Pods than the Deployment's desired two and creates a new Pod on its own. Compose's restart earlier reused the container and restarted the process; Kubernetes replaces the Pod instead. In both cases the new process starts with an empty counter.

**Check before continuing.** two application Pods `1/1` with zero restarts, one of them seconds old.

**Stop rule.** if the replacement has not reached `1/1` after 60 seconds, I stop and diagnose with `kubectl describe pod <new-pod>` rather than continue.

### Step 5 — Troubleshooting finding (2:40–3:40)

Purpose: break the Service selector, show healthy Pods with no endpoints, restore it, and show recovery measured by a successful request. The restore block is the README's "Restore and verify" block, unchanged.

**Enter** (VM, Terminal 2; failure group):

```
kubectl apply -f k8s/service-broken.yaml
kubectl get svc cs554-midterm -o jsonpath='{.spec.selector.app}'; echo
for attempt in $(seq 1 20); do
    count=$(kubectl get endpointslices -l kubernetes.io/service-name=cs554-midterm -o json \
      | jq '[.items[].endpoints[]? | select(.conditions.ready == true)] | length')
    if test "$count" -eq 0; then break; fi
    sleep 1
done
test "$count" -eq 0
failure_exit=0
kubectl exec client -- curl -sS --max-time 5 http://cs554-midterm/ || failure_exit=$?
printf 'Expected failed request exit: %s\n' "$failure_exit"
test "$failure_exit" -eq 7
kubectl get pods
kubectl get endpointslices -l kubernetes.io/service-name=cs554-midterm
```

**Observed in rehearsal:**

```
service/cs554-midterm configured
cs554-midterm-typo
curl: (7) Failed to connect to cs554-midterm port 80 after 0 ms: Could not connect to server
command terminated with exit code 7
Expected failed request exit: 7
NAME                             READY   STATUS    RESTARTS   AGE
client                           1/1     Running   0          24h
cs554-midterm-67c66d7b5d-2v5mx   1/1     Running   0          114m
cs554-midterm-67c66d7b5d-l2ffr   1/1     Running   0          21h
NAME                 ADDRESSTYPE   PORTS     ENDPOINTS   AGE
cs554-midterm-q55pw   IPv4          <unset>   <unset>     24h
```

**Enter** (VM, Terminal 2; restore group):

```
kubectl apply -f k8s/service.yaml
for attempt in $(seq 1 20); do
    count=$(kubectl get endpointslices -l kubernetes.io/service-name=cs554-midterm -o json \
      | jq '[.items[].endpoints[]? | select(.conditions.ready == true)] | length')
    if test "$count" -eq 2; then break; fi
    sleep 1
done
test "$count" -eq 2 || { echo 'Two ready endpoints did not return'; exit 1; }
kubectl get svc cs554-midterm -o jsonpath='{.spec.selector}'
kubectl get endpointslices -l kubernetes.io/service-name=cs554-midterm -o wide
kubectl describe svc cs554-midterm | grep -E 'Selector|Endpoints'
recovered=
for attempt in $(seq 1 20); do
    if kubectl exec client -- curl -sS --max-time 5 http://cs554-midterm/; then recovered=$attempt; break; fi
    sleep 1
done
echo "recovered on attempt ${recovered:-none}"
kubectl get pods -l app=cs554-midterm --show-labels
```

**Observed in rehearsal:**

```
service/cs554-midterm configured
{"app":"cs554-midterm"}
NAME                  ADDRESSTYPE   PORTS   ENDPOINTS               AGE
cs554-midterm-q55pw   IPv4          8080    10.42.0.17,10.42.0.19   24h
Selector:                 app=cs554-midterm
Endpoints:                10.42.0.17:8080,10.42.0.19:8080
curl: (7) Failed to connect to cs554-midterm port 80 after 0 ms: Could not connect to server
command terminated with exit code 7
{"service":"cs554-k8s","message":"Hello from Kubernetes inside the VM","hostname":"cs554-midterm-67c66d7b5d-l2ffr","requestCount":50}
recovered on attempt 2
NAME                              READY   STATUS    RESTARTS   AGE    LABELS
cs554-midterm-67c66d7b5d-2v5mx   1/1     Running   0          114m   app=cs554-midterm,pod-template-hash=67c66d7b5d,tier=web
cs554-midterm-67c66d7b5d-l2ffr   1/1     Running   0          21h    app=cs554-midterm,pod-template-hash=67c66d7b5d,tier=web
```

In this rehearsal the EndpointSlice already listed both backends when the first request was made, and that request was still refused; the second succeeded. That is the behavior the retry exists for: the EndpointSlice is updated before the node's forwarding rules are, so a successful request, not a populated EndpointSlice, is the evidence of recovery.

**Talking points.** Healthy Pods alone do not prove the Service works. Its selector is a label query, and the typo leaves it with no matching endpoints, so the request is refused while both Pods are running. After the correction the endpoints return at once, but the node's forwarding rules follow a moment later; the request is retried until it succeeds, here on attempt N.

**Check before continuing.** the JSON response and `recovered on attempt N` are on screen with a number, and both Pods carry `app=cs554-midterm`.

**Stop rule.** if the line reads `recovered on attempt none`, I stop the recording, keep the output, and run the restoration in Section 4; I do not claim recovery on camera.

### Step 6 — Closing (3:40–4:00)

Screen: the README's "Report and evidence" section. No command.

**Talking points.** The deployment and the experiments are captured in the evidence files, and the technical report explains the comparison and the OpenStack plan. The separate CS 554 readiness experiment is in `evidence/04-readiness-554.md` and section 6 of the report; it is not part of this on-camera demonstration.

# 4 Restoration and verification

### Restore after every run, finished or abandoned

```
cd ~/CS-554-MidTerm
kubectl apply -f k8s/service.yaml
kubectl rollout status deploy/cs554-midterm --timeout=120s
pkill -f 'kubectl.*port[-]forward' || true
```

Expected: `service/cs554-midterm unchanged` (or `configured`, if a run was abandoned during step 5), `successfully rolled out`, and no error from `pkill`.

### Verify the baseline

Preconditions P3, P4, and P5 are run again and must pass exactly as before: a healthy Compose service; `2/2`; two Pods `1/1` with zero restarts; selector `cs554-midterm`; two ready endpoints; both readiness files; no forwarder. The Pod names differ from before step 4; that is expected.

### Reset for another run (optional)

Nothing has to be reset: counters continue from where they are, and the talking points read the values on screen. If fresh counters are wanted, the following restarts the processes without changing a manifest or the Deployment:

```
cd ~/CS-554-MidTerm/compose && docker compose restart && cd ..
docker compose -f compose/compose.yaml ps
kubectl delete pod -l app=cs554-midterm
kubectl rollout status deploy/cs554-midterm --timeout=120s
```

Expected: Compose back to `Up ... (healthy)` with its next request counting `1`; two new application Pods `1/1` with new names and empty counters. The full set of preconditions is run again before recording.

### Completion criteria

The demonstration is complete when the recording shows all six steps with their expected results, the restoration has been run, the baseline has been verified, and the recording plays back with legible output and audible commentary. Compose, the cluster, and both replicas stay running afterwards.

## Appendix: recording notes

I record with Camtasia: a capture region that holds the terminals and the browser, microphone on, 1080p, exported as MP4. Every window stays inside that region. If Camtasia is unavailable, Snipping Tool's screen recorder is the second option and PowerPoint's Record Screen the third. The recording is uploaded unlisted, the link is opened in a private browser window to confirm it plays without a sign-in, and the link is then added to the README and the submission note.